

SENAI · M1S10 · Semana 10

Guia de HTML & CSS profissional

Classes CSS · Herança e cascata · Predominância · O mapa do HTML
profissional

23 documentos reunidos em um único PDF.

Guia de HTML & CSS profissional

23 documentos, da raiz ao avançado, para responder quatro perguntas: **o que cada classe faz, como funcionam herança e cascata, quem vence quando duas regras brigam e como se organiza um HTML de nível profissional.**

Como estudar

- 1 Leia na ordem.** Cada documento usa o que o anterior ensinou.
- 2 Abra o DevTools (F12)** em cada exemplo. Ver o CSS sendo aplicado vale mais que ler sobre ele.
- 3 Faça os exercícios do documento 16** antes de passar para a trilha D.
- 4 Volte ao mapa (documentos 17 a 19)** sempre que não souber que tag usar.

Trilha A — Classes CSS: nome, papel e função

Doc	Assunto
02	O que é uma classe e como nomeá-la
03	Classes de estrutura: container, wrapper, section
04	Classes de layout: flex, grid, row, col
05	Classes de componentes: botão, card, navbar, badge
06	Classes de texto e tipografia
07	Classes utilitárias (margem, cor, visibilidade)
08	Classes de estado e modificadores
09	Dicionário comercial: Bootstrap, Tailwind e CSS puro

Trilha B — Herança e efeito cascata

Doc	Assunto
10	O que é herança
11	O que herda, o que não herda, e inherit/initial/unset
12	O efeito cascata: as três perguntas
13	Cascata na prática com o DevTools

Trilha C — Predominância (especificidade)

Doc	Assunto
14	O placar de especificidade
15	Conflitos reais e como resolvê-los (sem !important)
16	Exercícios com gabarito (trilhas B e C)

Trilha D — O mapa do HTML profissional

Doc	Assunto
17	O mapa: esqueleto de uma página profissional
18	Árvore de decisão: div, main, section ou article?
19	Os elementos de página, um por um
20	Qual classe começa? A ordem de escrita do CSS
21	Nomenclatura (BEM) e organização de arquivos
22	Página completa anotada (Bikcraft)
23	Do básico ao avançado e checklist profissional

Regra de ouro do guia: HTML diz *o que a coisa é*. CSS diz *como ela parece*. A classe é a ponte entre os dois.

O que é uma classe CSS

Uma **classe** é uma etiqueta que você cola num elemento HTML para que o CSS consiga encontrá-lo e estilizá-lo. A classe não faz nada sozinha: ela só ganha poder quando existe uma regra CSS com o mesmo nome.

As três partes

```
<!-- HTML: o elemento recebe a etiqueta -->
<button class="botao">Enviar</button>

/* CSS: a regra procura a etiqueta (o ponto significa "classe") */
.botao {
  background: #1b4f72;
  color: #fff;
  padding: 10px 20px;
}
```

- **Nome:** `botao`. Você escolhe. O navegador não liga para o significado.
- **Ponto (.):** só aparece no CSS. No HTML escreve-se `class="botao"`, sem ponto.
- **Declarações:** o que está entre chaves, no formato `propriedade: valor;`.

Papel de uma classe

Toda classe responde a uma de quatro perguntas. Saber qual é a pergunta é o que separa um nome bom de um nome ruim.

Papel	Pergunta que responde	Exemplos
Estrutura	Onde esse bloco fica e qual a largura dele?	<code>container</code> , <code>wrapper</code> , <code>secao</code>
Layout	Como os filhos se organizam?	<code>flex</code> , <code>grid</code> , <code>row</code> , <code>col</code>
Componente	Que peça reutilizável é essa?	<code>botao</code> , <code>card</code> , <code>navbar</code>
Utilitária	Qual ajuste pequeno e único aplico?	<code>mt-2</code> , <code>text-center</code> , <code>hidden</code>
Estado/modificador	Em que situação ou variação está?	<code>ativo</code> , <code>is-open</code> , <code>botao--grande</code>

Cada um desses papéis tem um documento próprio (03 a 08).

Regras de sintaxe do nome

- Pode ter letras, números, hífen (-) e sublinhado (_).
- Não pode começar com número (`1coluna` não funciona; `coluna-1` funciona).
- Diferencia maiúscula de minúscula: `.Botao` e `.botao` são classes diferentes.
- Sem espaço. Espaço no atributo `class` **separa várias classes**.

Várias classes no mesmo elemento

```
<button class="botao botao--grande ativo">Comprar</button>
```

São três etiquetas no mesmo botão. As regras das três valem juntas. Esse é o mecanismo que permite combinar componente, variação e estado.

Classe, id ou tag?

Seletor	Exemplo	Reutiliza?	Use para
Tag	<code>p { }</code>	Em todos os <code>p</code>	Base global (documento 20)
Classe	<code>.card { }</code>	Sim, quantas vezes quiser	Quase todo o CSS
Id	<code>#topo { }</code>	Não, só um por página	Âncoras e JavaScript, não estilo

Padrão profissional: estilize com **classes**. Use `id` para links de âncora (`href="#contato"`) e para o JavaScript. O motivo está no documento 14: o id pesa tanto que atrapalha quem vem depois.

Nome bom e nome ruim

Evite	Por quê	Prefira
<code>.azul</code>	Descreve a aparência. Se a marca mudar de cor, o nome mente.	<code>.botao-primario</code>
<code>.caixa2</code>	Não diz nada.	<code>.card-produto</code>
<code>.div-esquerda</code>	Descreve posição e a tag.	<code>.coluna-dados</code>
<code>.Menu_Principal</code>	Mistura estilos de escrita.	<code>.menu-principal</code>

Regra: o nome diz **o que a coisa é** (função), não **como ela se parece**. Escreva em minúsculas, com hífen entre as palavras (*kebab-case*).

Classes de estrutura

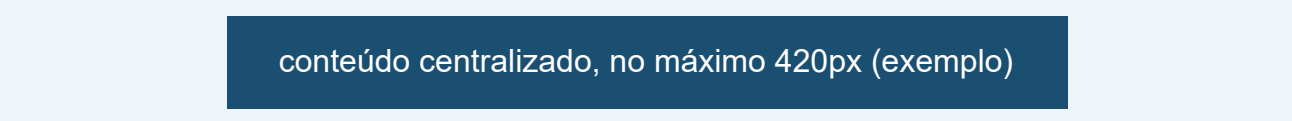
São as classes que definem **largura, centralização e ritmo vertical** da página. Quase todo site profissional começa por elas. A mais famosa é a `container`.

container

Função: limitar a largura do conteúdo e centralizá-lo. Sem ele, em monitor grande o texto estica de ponta a ponta e fica ilegível.

```
.container {  
  width: 100%;  
  max-width: 1100px; /* nunca passa disso */  
  margin: 0 auto; /* auto dos lados = centraliza */  
  padding: 0 16px; /* respiro nas bordas do celular */  
}
```

COMO SE COMPORTA

Um diagrama que mostra um retângulo cinza claro representando o conteúdo, centralizado dentro de um retângulo azul escuro maior, que representa o container. O texto "conteúdo centralizado, no máximo 420px (exemplo)" está escrito no retângulo azul escuro.

conteúdo centralizado, no máximo 420px (exemplo)

Detalhe importante: o fundo pode ocupar a tela inteira (na `section`) enquanto o conteúdo fica no `container` dentro dela. Veja o padrão abaixo.

O padrão "faixa + container"

```
<section class="secao secao-escura"> <!-- faixa de fundo: 100% da largura -->  
  <div class="container"> <!-- conteúdo: largura limitada -->  
    <h2>Nossas bikes</h2>  
    ...  
  </div>  
</section>
```

É o padrão mais comum do mercado. A faixa pinta a tela toda; o container alinha o conteúdo.

Tabela das classes de estrutura

Classe	Papel	CSS típico
<code>container</code>	Limita e centraliza a largura	<code>max-width; margin: 0 auto</code>
<code>container-fluid</code>	Container que ocupa 100% (Bootstrap)	<code>width: 100%</code>
<code>wrapper</code> / <code>wrap</code>	Envelope que agrupa um bloco maior (às vezes a página inteira)	<code>min-height: 100vh</code> ou <code>max-width</code>
<code>secao</code> / <code>section</code>	Faixa horizontal com espaçamento vertical	<code>padding: 64px 0</code>
<code>hero</code>	Primeira faixa, de destaque	<code>min-height</code> , imagem de fundo
<code>conteudo</code> / <code>content</code>	Miolo da página ou de um bloco	depende do projeto
<code>sidebar</code> / <code>lateral</code>	Coluna lateral	<code>width</code> fixa + <code>flex-shrink: 0</code>
<code>cabecalho</code> / <code>header</code> , <code>rodape</code> / <code>footer</code>	Topo e base	fundo, padding, às vezes <code>position</code>

Container x wrapper: qual a diferença?

Na prática os dois fazem parte da mesma família, e equipes diferentes usam nomes diferentes. A convenção mais útil para a sala:

- **container** = controla **largura horizontal** do conteúdo.
- **wrapper** = embrulha um grupo para dar-lhe **comportamento próprio** (altura mínima, posição relativa, overflow).

O importante é **escolher uma convenção e repetir** no projeto inteiro.

Pergunta de revisão: por que o `container` usa `max-width` e não `width`? Resposta: `width` fixa estoura o celular; `max-width` encolhe junto com a tela e só limita o teto.

Classes de layout

Classes de layout dizem **como os filhos de um elemento se organizam**: em linha, em coluna, em grade. A regra mental: a classe vai no **pai**, e quem se organiza são os **filhos**.

Flexbox: organizar em uma direção

```
.flex { display: flex; }  
.flex-coluna { flex-direction: column; }  
.centralizado { justify-content: center; align-items: center; }  
.espaco-entre { justify-content: space-between; }  
.quebra { flex-wrap: wrap; }  
.gap { gap: 16px; }
```

FLEX + GAP + ESPACO-ENTRE



Quando usar: menu, barra de botões, cabeçalho, centralizar algo, linha de cards que podem quebrar.

Grid: organizar em linhas e colunas ao mesmo tempo

```
.grid { display: grid; gap: 16px; }  
.grid-3 { grid-template-columns: repeat(3, 1fr); }  
.grid-auto { grid-template-columns: repeat(auto-fit, minmax(240px, 1fr)); }
```

GRID-AUTO: REDIMENSIONE A JANELA E VEJA AS COLUNAS MUDAREM SOZINHAS



Quando usar: galeria, catálogo de produtos, layout de página inteira, qualquer coisa em "grade".

Regra prática: uma dimensão (linha *ou* coluna) → **flex**. Duas dimensões (linhas e colunas) → **grid**.

row e col (Bootstrap) e colunas do projeto

Em frameworks, o layout vira vocabulário pronto:

```
<div class="container">
  <div class="row">
    <div class="col-md-6">Dados</div>
    <div class="col-md-6">Formulário</div>
  </div>
</div>
```

- `row` : a linha (por baixo é `display: flex; flex-wrap: wrap`).
- `col-md-6` : "a partir da tela média, ocupe 6 de 12 partes (metade)".

Em CSS puro você cria o mesmo com nome por função, como a `coluna-dados` e a `coluna-formulario` da página de contato da Bikcraft.

Tabela das classes de layout

Classe	O que faz	Vai em
<code>flex</code> / <code>d-flex</code>	Liga o flexbox	o pai
<code>grid</code> / <code>d-grid</code>	Liga o grid	o pai
<code>row</code>	Linha de colunas	o pai
<code>col</code> , <code>col-6</code>	Coluna e sua largura	o filho
<code>gap</code>	Espaço entre os filhos	o pai
<code>justify-*</code> , <code>items-*</code>	Alinha no eixo principal / cruzado	o pai
<code>stack</code>	Pilha vertical com espaço regular	o pai

Erro clássico: colocar `display: flex` no filho e esperar que os irmãos se alinhem. O flex só organiza os **filhos diretos** de quem o recebe.

Classes de componentes

Um **componente** é uma peça reutilizável da interface, com visual completo: botão, card, menu, formulário. A classe de componente entrega a peça **inteira**. É a classe mais comum em projetos reais.

Anatomia de um componente

Um componente tem um **bloco** (a peça) e **partes** (o que existe dentro dela). Os nomes das partes começam pelo nome do bloco.

botao

```
.botao {  
  display: inline-block;  
  padding: 10px 20px;  
  border-radius: 6px;  
  background: #1b4f72; color: #fff;  
  text-decoration: none; font-weight: bold;  
}  
.botao:hover { background: #123049; }
```

RESULTADO



Serve para `<button>` e para `<a>` que parece botão. Escolha a tag pelo **comportamento** (ação → button; navegação → a) e a classe pelo **visual**.

card

```
<article class="card">  
    
  <h3 class="card-titulo">Urbana</h3>  
  <p class="card-texto">Leve e prática.</p>  
  <a class="botao" href="#">Ver</a>  
</article>
```

RESULTADO

Urbana

Leve e prática.

Ver

Note os nomes: `card-imagem` , `card-titulo` , `card-texto` . Quem lê o HTML entende o dono de cada pedaço.

badge

RESULTADO

Entrega rápida novo

Selo pequeno que rotula algo: "novo", "promoção", contador de notificações.

Tabela dos componentes mais comuns do mercado

Classe	Função	Tag usada
<code>botao</code> / <code>btn</code>	Ação ou chamada principal	<code>button</code> , <code>a</code>
<code>card</code>	Cartão de produto, notícia, perfil	<code>article</code> , <code>div</code>
<code>navbar</code> / <code>menu</code>	Barra de navegação	<code>nav</code>
<code>logo</code>	Marca no topo	<code>a</code> , <code>span</code> , <code>img</code>
<code>hero</code>	Faixa de destaque da página	<code>section</code>
<code>badge</code> / <code>tag</code>	Selo pequeno de rótulo	<code>span</code>
<code>alert</code> / <code>aviso</code>	Mensagem de sucesso, erro, atenção	<code>div</code> com <code>role="alert"</code>
<code>campo</code> / <code>form-group</code>	Rótulo + entrada de formulário	<code>div</code> com <code>label</code> e <code>input</code>
<code>modal</code>	Janela sobre a página	<code>dialog</code> , <code>div</code>
<code>lista</code> , <code>tabela</code>	Listagem e dados	<code>ul</code> , <code>table</code>

Teste do bom componente: se eu copiar este HTML para outra página, ele continua bonito sem eu mexer no CSS? Se sim, é um componente de verdade.

Classes de texto e tipografia

Tipografia é o que mais pesa na aparência de um site. As classes de texto controlam **tamanho, peso, alinhamento e cor**. Há uma regra que vale para todas: a **tag diz o nível** do texto; a **classe diz o visual**.

Tag para significado, classe para aparência

Errado: `<p class="titulo-grande">` só para parecer título. Leitor de tela e Google não reconhecem como título.

Certo: `<h2 class="titulo-secao">` . A tag dá o nível correto; a classe ajusta o tamanho.

Também vale o contrário: se um `<h3>` precisa parecer pequeno, mantenha o `h3` e reduza com classe. Nunca troque o nível só pelo tamanho.

Classes de texto mais usadas

Classe	Função	CSS típico
<code>titulo</code> , <code>titulo-secao</code>	Título de página ou seção	<code>font-size: 2rem; line-height: 1.2</code>
<code>subtitulo</code>	Linha de apoio ao título	<code>font-size: 1.1rem; color: #546e7a</code>
<code>lead</code> / <code>intro</code>	Parágrafo de abertura, maior que o normal	<code>font-size: 1.15rem</code>
<code>texto-pequeno</code> / <code>small</code>	Legendas, avisos legais	<code>font-size: .85rem</code>
<code>texto-destaque</code>	Palavra ou frase que chama atenção	<code>font-weight: bold; color</code>
<code>text-center</code>	Centraliza o texto	<code>text-align: center</code>
<code>text-uppercase</code>	Tudo em maiúsculas	<code>text-transform: uppercase</code>
<code>text-truncate</code>	Corta com "..." se não couber	<code>overflow: hidden; text-overflow: ellipsis; white-space: nowrap</code>

Propriedades de texto que precisam estar no seu vocabulário

- `font-family` : a fonte (sempre com alternativas: `"Inter"`, `Arial`, `sans-serif`).
- `font-size` : use `rem` para respeitar a configuração do usuário.
- `font-weight` : `400` normal, `700` negrito.
- `line-height` : altura da linha; `1.5` a `1.7` é ótimo para parágrafos.
- `text-align` , `letter-spacing` , `text-decoration` .

Onde declarar a fonte? Uma vez só, no `body` . Todo o resto **herda** (documento 10). Quem repete `font-family` em dez classes está lutando contra o CSS.

Hierarquia de título: a "escada"

```
<h1>  um por página (o assunto da página)
<h2>  cada seção
  <h3>  subtítulos dentro da seção
    <h4>  detalhes (raro)
```

Não pule degraus (de `h1` direto para `h4`) e use **um único h1** por página.

Classes utilitárias

Uma classe **utilitária** faz **uma única coisa**, sempre a mesma, em qualquer lugar. É o contrário do componente: não entrega uma peça pronta, entrega um ajuste fino.

Exemplos clássicos

```
.mt-2 { margin-top: 8px; }
.mb-4 { margin-bottom: 24px; }
.p-3 { padding: 16px; }
.text-center { text-align: center; }
.hidden { display: none; }
.w-100 { width: 100%; }
.sr-only { /* escondido da tela, mas lido por leitor de tela */
  position: absolute; width: 1px; height: 1px;
  overflow: hidden; clip: rect(0 0 0 0); white-space: nowrap;
}
```

Por que existem

Sem utilitárias, cada ajuste de 8px viraria uma classe nova. Com elas, você resolve um detalhe sem inventar nome:

```
<section class="secao text-center mb-4">...</section>
```

Escala de espaçamento

Profissionais não usam números aleatórios; usam uma **escala**. Assim a página inteira respira no mesmo ritmo.

Sufixo	Valor	Uso típico
-1	4px	Espaço mínimo
-2	8px	Entre ícone e texto
-3	16px	Padding de componente
-4	24px	Entre blocos
-5	48px	Entre seções

Leitura do nome: `mt` = margin-top, `mb` = bottom, `ms` / `me` = start/end, `mx` = eixo horizontal, `my` = vertical; `p...` = padding.

Utilitárias mais comuns

Grupo	Exemplos	Efeito
Espaço	<code>m-0</code> , <code>mt-3</code> , <code>px-4</code>	margem e padding
Exibição	<code>hidden</code> , <code>d-none</code> , <code>block</code>	<code>display</code>
Tamanho	<code>w-100</code> , <code>h-100</code> , <code>max-w-full</code>	largura e altura
Texto	<code>text-center</code> , <code>fw-bold</code>	alinhamento e peso
Cor	<code>text-primary</code> , <code>bg-dark</code>	cor de texto e fundo
Acessibilidade	<code>sr-only</code>	só leitor de tela

Utilitária x componente: quando usar cada uma

	Componente	Utilitária
Faz	Uma peça inteira	Um ajuste único
Nome	Substantivo: <code>card</code>	Ação curta: <code>mt-3</code>
Bom para	O que se repete igual	Exceções pontuais
Risco	Virar CSS gigante	HTML poluído de classes

Cuidado: utilitárias costumam ser escritas **depois** de tudo no CSS (documento 20) para vencer o componente em caso de empate. É assim que `mt-0` consegue anular a margem de um `card`.

Tailwind CSS leva essa ideia ao limite: *todo* o visual vem de utilitárias (`flex p-4 text-center`). É a abordagem "utility-first" citada na atividade da biblioteca.

Classes de estado e modificadores

Duas ideias parecidas, mas diferentes. **Modificador** = uma *variação fixa* do componente (grande, perigo). **Estado** = uma *situação que muda* com o uso (aberto, ativo, desabilitado).

Modificador: a variação do componente

O componente base fica sozinho e os modificadores só **alteram o que for diferente**.

```
.botao { padding: 8px 16px; background: #cfd8dc; }  
.botao--primario { background: #1b4f72; color: #fff; }  
.botao--perigo { background: #c0392b; color: #fff; }  
.botao--grande { padding: 14px 28px; }
```

```
<button class="botao">Cancelar</button>  
<button class="botao botao--primario">Salvar</button>  
<button class="botao botao--perigo botao--grande">Excluir</button>
```

RESULTADO

Cancelar

Salvar

Excluir

O **duplo hífen** (--) é a convenção BEM (documento 21) para "modificador". Quem lê sabe que `botao--grande` depende de `botao`.

Estado: a situação do momento

O estado muda por causa de uma ação do usuário, normalmente com JavaScript ligando e desligando a classe.

```
.menu { display: none; }  
.menu.is-open { display: block; } /* JS adiciona e remove is-open */  
  
.botao.ativo { outline: 3px solid gold; }
```

BOTÃO "ATIVO"

Comprar

Nomes de estado comuns

Classe	Significa
<code>ativo</code> / <code>active</code>	Item atual (aba, link da página atual)
<code>is-open</code> / <code>aberto</code>	Menu, modal ou acordeão aberto
<code>is-hidden</code> / <code>oculto</code>	Escondido no momento
<code>is-loading</code>	Carregando
<code>is-invalid</code> / <code>erro</code>	Campo com erro
<code>disabled</code>	Desabilitado

O prefixo `is-` / `has-` avisa à equipe: "esta classe é controlada por script, não escreva à mão no HTML".

Pseudoclasses: os estados nativos do CSS

Alguns estados o navegador já conhece. Não precisa de classe nem JavaScript.

Pseudoclasse	Quando vale
<code>:hover</code>	Mouse em cima
<code>:focus</code> , <code>:focus-visible</code>	Elemento selecionado pelo teclado
<code>:active</code>	No instante do clique
<code>:disabled</code> , <code>:checked</code>	Campo desabilitado, caixa marcada
<code>:first-child</code> , <code>:nth-child(2)</code>	Posição entre os irmãos

Não remova o `:focus` (`outline: none`) sem colocar outro indicador no lugar. Quem navega por teclado fica perdido.

Dicionário comercial: Bootstrap, Tailwind e CSS puro

No mercado você vai encontrar três "sotaques" de classes. Este documento é a **tabela de tradução**: o mesmo resultado escrito nos três estilos.

Três filosofias

Bootstrap

Componentes prontos. `btn btn-primary` entrega o botão inteiro. Rápido, padronizado, visual conhecido.

Tailwind

Utility-first. Cada classe é uma propriedade. Controle fino, HTML cheio de classes.

CSS puro

Você nomeia tudo. `botao botao--primario`. Máximo controle, mais trabalho. É o que usamos na Bikcraft.

Tabela de tradução

Quero...	CSS puro	Bootstrap	Tailwind
Limitar e centralizar a largura	<code>container</code>	<code>container</code>	<code>mx-auto max-w-6xl px-4</code>
Ligar flexbox	<code>flex</code>	<code>d-flex</code>	<code>flex</code>
Centralizar com flex	<code>centralizado</code>	<code>d-flex justify-content-center align-items-center</code>	<code>flex justify-center items-center</code>
Espaço entre filhos	<code>gap</code>	<code>gap-3</code>	<code>gap-4</code>
Metade da largura	<code>coluna-metade</code>	<code>col-md-6</code>	<code>md:w-1/2</code>
Botão principal	<code>botao botao--primario</code>	<code>btn btn-primary</code>	<code>bg-blue-700 text-white px-4 py-2 rounded</code>
Card	<code>card</code>	<code>card</code>	<code>border rounded-lg p-4 shadow</code>
Texto centralizado	<code>text-center</code>	<code>text-center</code>	<code>text-center</code>
Margem de cima	<code>mt-3</code>	<code>mt-3</code>	<code>mt-4</code>
Esconder	<code>hidden</code>	<code>d-none</code>	<code>hidden</code>
Esconder só no celular	media query	<code>d-none d-md-block</code>	<code>hidden md:block</code>
Só leitor de tela	<code>sr-only</code>	<code>visually-hidden</code>	<code>sr-only</code>

Vocabulário de breakpoints

Prefixo	Começa em	Dispositivo típico
(nenhum)	0	Celular (mobile-first)
sm	576px (BS) / 640px (TW)	Celular grande
md	768px	Tablet
lg	992px (BS) / 1024px (TW)	Notebook
xl	1200px (BS) / 1280px (TW)	Desktop

Todos seguem **mobile-first**: `md:w-1/2` significa "de 768px para cima".

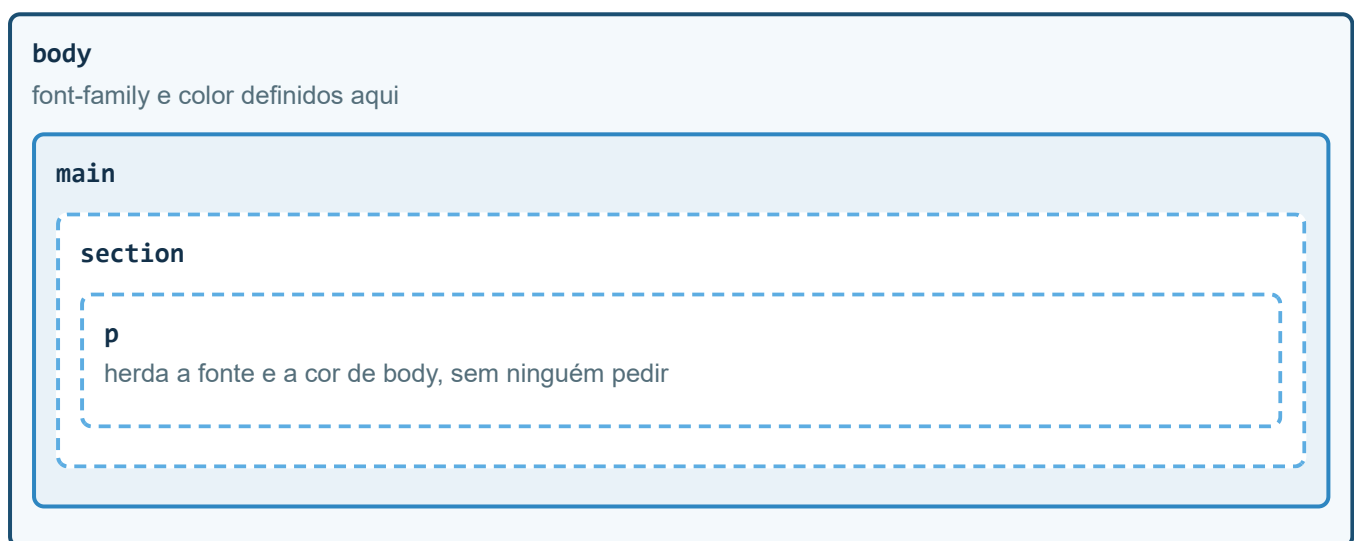
Por que aprender CSS puro primeiro? Porque o `row` do Bootstrap é `flex + flex-wrap`, o `col-md-6` é `flex: 0 0 50%` dentro de uma media query, e o `p-4` do Tailwind é `padding: 1rem`. Quem entende a base lê qualquer framework.

O que é herança

Herança é quando um elemento **filho recebe automaticamente** certos valores do elemento **pai**, sem que você escreva nada para ele. É o motivo de bastar declarar a fonte uma vez no `body`.

A árvore do HTML

Todo HTML é uma árvore de elementos aninhados. Quem está dentro é **filho**; quem envolve é **pai**; quem está acima de todos é **ancestral**.



Exemplo mínimo

```
body { font-family: Arial, sans-serif; color: #263238; }
```

```
/* Não existe regra de fonte para h1, p, li, a... e mesmo assim todos usam Arial */
```

PAI COM COR E FONTE; FILHOS SEM REGRA NENHUMA

Eu sou o pai (Georgia, azul).

Eu sou um **p** filho, sem CSS próprio: herdei a fonte e a cor.

- Item de lista, também herdou.

Por que a herança existe

- **Economia:** uma declaração no ancestral vale para centenas de filhos.
- **Consistência:** a página inteira fala a mesma "língua" visual.
- **Manutenção:** mudar a fonte do site é mexer em **uma linha**.

Herança x cascata: não confunda

	Herança	Cascata
Pergunta	"De quem este elemento recebe o valor?"	"Quando duas regras dizem coisas diferentes, qual vale?"
Direção	Do pai para o filho	Entre regras que atingem o mesmo elemento
Documento	10 e 11	12, 13 e 14

Analogia: herança é o **sobrenome da família**: você recebe sem pedir. Se você disser "meu nome é outro", o que **você mesmo declarou** vence o que herdou. Uma regra direta no filho sempre ganha da herança.

```
body { color: #263238; }  
.aviso { color: #c0392b; } /* o .aviso não herda mais a cor: usa a própria */
```

Atenção: nem tudo é herdado. Borda, margem e fundo, por exemplo, **não** passam do pai para o filho. É o assunto do próximo documento.

O que herda e o que não herda

A regra geral: **propriedades de texto são herdadas**; propriedades de caixa (tamanho, borda, fundo, espaço) **não são**. Faz sentido: se toda `div` herdasse a borda do pai, a página viraria uma cebola de molduras.

Tabela de referência

✓ Herdam

- `color`
- `font-family`, `font-size`, `font-weight`, `font-style`
- `line-height`
- `text-align`, `letter-spacing`
- `visibility`
- `cursor`
- `list-style`

✗ Não herdam

- `margin`, `padding`
- `border`
- `background`
- `width`, `height`
- `display`, `position`
- `overflow`
- `box-shadow`, `opacity`

PAI COM BORDA + COR VERDE. QUEM HERDOU O QUÊ?

Pai: tem borda e texto verde.

Filho **p**: o texto ficou **verde** (herdou), mas **sem borda** (não herda).

Casos que pegam todo mundo

1. Links não herdam a cor do pai

O navegador dá ao `<a>` uma cor própria (azul). Para o link acompanhar o texto ao redor:

```
a { color: inherit; text-decoration: none; }
```

2. Campos de formulário não herdam a fonte

`input`, `button`, `select` e `textarea` usam a fonte do sistema. Reset profissional:

```
input, button, select, textarea { font: inherit; }
```

3. line-height herda como número, não como valor calculado

`line-height: 1.5` (sem unidade) é recalculado em cada filho conforme o tamanho da fonte dele. `line-height: 24px` passa fixo. Prefira o número sem unidade.

Palavras-chave que controlam a herança

Valor	O que faz	Exemplo
<code>inherit</code>	Força herdar do pai, mesmo em propriedade que normalmente não herda	<code>border: inherit;</code> , <code>color: inherit;</code>
<code>initial</code>	Volta ao valor padrão da especificação CSS	<code>display: initial;</code>
<code>unset</code>	Se a propriedade herda → <code>inherit</code> ; se não → <code>initial</code>	<code>all: unset;</code>
<code>revert</code>	Volta ao estilo padrão do navegador	<code>color: revert;</code>

Receita profissional para a base do site:

```
body { font-family: Arial, sans-serif; line-height: 1.6; color: #263238; }  
a { color: inherit; }  
input, button, textarea, select { font: inherit; }
```

Três linhas de herança bem usada evitam dezenas de classes repetidas.

O efeito cascata

CSS significa **Cascading** Style Sheets: folhas de estilo **em cascata**. Cascata é o algoritmo que decide **qual regra vence** quando várias regras atingem o mesmo elemento e dizem coisas diferentes sobre a mesma propriedade.

O problema que a cascata resolve

```
p { color: blue; }  
p { color: red; }  
.aviso { color: green; }  
  
<p class="aviso">De que cor eu fico?</p>
```

Três regras, três cores, um parágrafo. O navegador não pode obedecer às três. A cascata é o desempate. (Resposta: **verde**, e você vai saber por quê já já.)

As três perguntas, nesta ordem

O navegador pergunta, na sequência, e para assim que alguém vence:

1 Origem e importância. De onde vem a regra: estilo do navegador, do autor (você) ou do usuário? Tem `!important` ?

2 Especificidade. Qual seletor é *mais específico*? (Detalhado no documento 14.)

3 Ordem no código. Se empatou nas duas, **a última escrita vence**.

1. Há `!important` ou origens diferentes?

↓ não, empatou

2. Um seletor é mais específico?

↓ não, empatou

3. Qual aparece por último no CSS?



Essa vence

1. Origem

Origem	Quem escreve	Peso
Navegador (user agent)	Chrome, Firefox... (azul do link, margem do <code>body</code>)	Menor
Autor	Você , no seu arquivo .css	Maior que a do navegador
Usuário	Preferências de acessibilidade	Entre as duas

Por isso o seu CSS sempre vence o estilo padrão do navegador, mesmo com seletores simples.

3. Ordem: a última vence (em caso de empate)

```
.ex-ordem { color: #1b4f72; } /* azul */  
.ex-ordem { color: #c0392b; } /* vermelho – vem depois, vence */
```

DUAS REGRAS COM A MESMA ESPECIFICIDADE: VERMELHO (A ÚLTIMA)

Este texto aparece vermelho.

Ordem também vale entre arquivos

```
<link rel="stylesheet" href="base.css">  
<link rel="stylesheet" href="pagina.css"> <!-- vem depois: ganha os empates -->
```

Por isso o reset e a base vêm primeiro, e o CSS da página, depois.

Por que "cascata"?

Como uma cachoeira, o estilo desce de cima (navegador) para baixo (seu CSS, regra por regra), e **o que chega por último ao elemento é o que ele mostra**. Cada regra

"sobrescreve" a anterior apenas nas propriedades em que discordam; as outras continuam valendo.

```
p      { color: blue; font-size: 18px; }  
.aviso { color: green; }  
/* Resultado em <p class="aviso">: cor verde (do .aviso), tamanho 18px (do p).  
   As regras se SOMAM nas propriedades diferentes e só brigam nas iguais. */
```

Resumo em uma frase: a cascata combina todas as regras que atingem o elemento; onde elas discordam, vence a de **maior importância** → **maior especificidade** → **mais tarde no código**.

Cascata na prática com o DevTools

Teoria entra de verdade quando você **vê** a cascata. O DevTools mostra, para cada elemento, **todas** as regras que o atingem, quais venceram e quais foram riscadas.

Passo a passo no navegador

- 1 Abra a página e aperte **F12** (ou clique com o botão direito → Inspecionar).
- 2 Clique na seta do canto da janela e depois no elemento que quer investigar.
- 3 Na aba **Elementos**, olhe o painel **Estilos** (Styles).
- 4 Leia de cima para baixo: **regras mais fortes ficam em cima**.
- 5 Propriedade riscada = perdeu a briga. Passe o mouse e o navegador mostra por quê.
- 6 Mais abaixo, "*Inherited from ...*" mostra o que veio por herança.
- 7 Aba **Computed** (Computado): o valor **final** de cada propriedade.

Laboratório 1: descubra o vencedor

```
.t1 { color: #c0392b; }  
p { color: #1e8449; }  
  
<p class="t1">Qual cor?</p>
```

INSPECIONE ESTE PARÁGRAFO COM F12

Qual cor? (vermelha: classe vence tag)

No DevTools você verá `p { color: green }` riscada.

Laboratório 2: regras se somam

```
.t2 { color: #1b4f72; font-size: 1.2rem; }  
.t2.negrito { font-weight: bold; }
```

DUAS REGRAS, ZERO CONFLITO: AMBAS VALEM

Azul, grande e negrito.

Nenhuma foi riscada, porque nenhuma discorda da outra.

!important: o botão vermelho

```
p { color: red !important; }
```

`!important` pula as etapas 2 e 3 da cascata. Nada que o autor escrever sem ele consegue vencer.

Por que evitar: a única forma de vencer um `!important` é outro `!important` mais forte. A manutenção vira uma corrida armamentista. Se você precisou dele, quase sempre o problema é de especificidade (documento 15).

Usos legítimos: utilitárias que *precisam* sempre vencer (`.hidden { display: none !important; }`) e sobrescrever CSS de terceiros que você não controla.

O atributo style no HTML

```
<p style="color: red">
```

Estilo **inline** vence qualquer seletor de arquivo, sem precisar de `!important`. Por isso se evita no dia a dia: ele escapa da organização do CSS e só pode ser vencido por `!important`.

Checklist de investigação

- ☐ O seletor realmente encontra o elemento? (erro de digitação, ponto esquecido)

- ☐ O arquivo CSS foi carregado? (aba Network / link quebrado)
- ☐ Outra regra mais específica sobrescreve a minha?
- ☐ Duas regras empatam e a última do arquivo vence?
- ☐ A propriedade herda? Se não, preciso declará-la no próprio elemento.

O placar de especificidade

Especificidade é a nota que o navegador dá a cada seletor. Entre duas regras que brigam, **a de maior nota vence**, independentemente da ordem no arquivo. É assim que se decide a predominância entre classes, ids e tags.

O placar tem 4 colunas

Pense num placar **A – B – C – D**. Compara-se coluna por coluna, da esquerda para a direita. **A primeira coluna que tiver número maior ganha**, e as demais nem são olhadas.

A	B	C	D
estilo inline style="..."	ids #topo	classes, atributos, pseudoclasses .card [type] :hover	tags e pseudoelementos p h1 ::before

Tabela de pontuação

Seletor	A	B	C	D	Leitura
p	0	0	0	1	0-0-0-1
p.aviso	0	0	1	1	0-0-1-1
.card	0	0	1	0	0-0-1-0
.card .titulo	0	0	2	0	0-0-2-0
nav ul li a	0	0	0	4	0-0-0-4
a:hover	0	0	1	1	0-0-1-1
#topo	0	1	0	0	0-1-0-0
#topo .menu a	0	1	1	1	0-1-1-1
style="..."	1	0	0	0	1-0-0-0

Como comparar: o exemplo mais importante da trilha

Uma classe vence **onze tags empilhadas**, porque a coluna C pesa mais do que qualquer quantidade na coluna D:

```
.botao      { color: red; }      /* 0-0-1-0 */
html body main section div p a { color: blue; } /* 0-0-0-7 */
/* Vence .botao: 1 na coluna C ganha de 7 na coluna D */
```

A mesma lógica vale um nível acima: **um id vence cem classes**.

MESMO ELEMENTO, CLASS + ID. A ORDEM NO CSS É .X1 DEPOIS #X1; QUEM APARECE AZUL?

Azul: o id (0-1-0-0) vence a classe (0-0-1-0), mesmo se a classe viesse por último.

O que NÃO conta pontos

- O seletor universal `*`.
- Combinadores: espaço, `>`, `+`, `~`.
- `:where( )` → sempre zero. `:is( )` e `:not( )` → valem o item mais forte lá dentro.

Empate: volta a regra da ordem

Se o placar é igual nas quatro colunas, **vence a regra que vem por último** no código (documento 12).

Pegadinha de herança: um valor **herdado** perde para **qualquer** regra direta no elemento, mesmo a de especificidade `0-0-0-1`. Herança não entra no placar: ela só preenche o que ninguém declarou.

Conflitos reais e como resolvê-los

Esta é a parte prática: "o meu CSS não funciona". Quase sempre a causa é um desses cinco casos. Cada um tem a **solução profissional**, que raramente é usar `!important`.

Caso 1: o link que não muda de cor

```
.menu a { color: #fff; } /* 0-0-1-1 */
.destaque { color: gold; } /* 0-0-1-0 - perde! */

<nav class="menu"><a class="destaque" href="#">Contato</a></nav>
```

Diagnóstico: `.menu a` pesa 0-0-1-1; `.destaque` pesa 0-0-1-0. A mais pesada vence, mesmo vindo antes.

Solução profissional: igualar ou superar o peso **com seletor**, não com força bruta.

```
.menu .destaque { color: gold; } /* 0-0-2-0 - vence */
```

Caso 2: o seletor muito longo

```
body main section.hero div.container h1.titulo { ... } /* 0-0-3-4 */
```

Quem for sobrescrever precisará de um seletor ainda maior. Cada seletor longo "contamina" o projeto.

Solução: uma classe, só uma, direto no elemento.

```
.hero-titulo { ... } /* 0-0-1-0 */
```

Caso 3: o id no CSS

```
#topo { background: navy; } /* 0-1-0-0: nenhuma classe consegue vencer */
```

Solução: troque por classe. Mantenha o `id` para âncora e JavaScript.

Caso 4: a ordem dos arquivos

Seu componente não pega porque o CSS do framework está **depois** do seu.

```
<link rel="stylesheet" href="meu.css">
<link rel="stylesheet" href="bootstrap.css"> <!-- vem depois: ganha os empates -->
```

Solução: framework primeiro, o seu depois.

Caso 5: modificador que não "pega"

```
.botao--perigo { background: red; } /* escrito ANTES de .botao */
.botao         { background: gray; } /* mesmo peso, vem depois: vence */
```

Solução: base primeiro, modificadores e estados depois. Documento 20.

Resumo das armas, da mais saudável à pior

#	Ferramenta	Avaliação
1	Reordenar o CSS	✅ Grátis, não cria dívida
2	Trocar para uma classe simples	✅ Melhor correção de raiz
3	Acrescentar uma classe ao seletor (<code>.menu</code> <code>.destaque</code>)	🟡 Aceitável, cuidado com a escalada
4	<code>:where()</code> para zerar o peso de base	🟡 Avançado e elegante
5	<code>!important</code>	🔴 Último recurso
6	<code>id</code> ou <code>style inline</code>	🔴 Evitar

Predominância semântica: o princípio por trás de tudo

Para a pergunta *"que classe predomina?"*, a resposta de projeto é uma **hierarquia de poder deliberada**:

Tags (base)

0-0-0-1: o piso. Define o padrão do site.

Classes de componente

0-0-1-0: a regra geral de cada peça.

Modificadores e estados

vêm depois e vencem o componente.

Utilitárias

por último: precisam vencer tudo (às vezes com !important).

Quanto mais "específico em significado" o ajuste, mais tarde ele entra no CSS. Assim, a vontade do desenvolvedor ("este botão é perigoso", "esta margem é zero") sempre prevalece sobre o padrão genérico, **sem recorrer a briga de peso**.

Mantra: baixa especificidade, ordem organizada. Se você escreve seletores de peso 0-0-1-0 na ordem certa, nunca precisará lutar com a cascata.

Exercícios: herança, cascata e especificidade

Faça **sem abrir o navegador**, no papel ou em dupla. Depois confira clicando em "Ver gabarito" e confirme no DevTools.

Parte 1 — Herança

Exercício 1

```
body { color: navy; border: 2px solid black; }  
  
<body><p>Texto</p></body>
```

O `p` terá cor azul-marinho? E borda?

▼ Ver gabarito

Cor: **sim** (`color` herda). Borda: **não** (`border` não herda). A borda aparece só no `body`.

Exercício 2

```
body { color: navy; }  
  
<body><a href="#">Link</a></body>
```

De que cor é o link? Como faria para ele acompanhar o texto?

▼ Ver gabarito

O link fica na **cor padrão do navegador (azul)**, pois o navegador define uma cor própria para `a`. Solução: `a { color: inherit; }`.

Parte 2 — Cascata (ordem)

Exercício 3

```
.titulo { color: red; }  
.titulo { color: green; }  
<h1 class="titulo">Oi</h1>
```

▼ Ver gabarito

Verde. Empate de especificidade; vence a última.

Exercício 4

```
p { color: blue; font-size: 20px; }  
p { color: red; }  
<p>Oi</p>
```

Cor e tamanho?

▼ Ver gabarito

Cor **vermelha** (a última vence), tamanho **20px** (ninguém discordou; a regra anterior continua valendo).

Parte 3 — Especificidade

Exercício 5: calcule o placar

Seletor	Placar (A-B-C-D)
<code>h2</code>	
<code>.card h2</code>	
<code>#lojas .card h2</code>	
<code>a:hover</code>	
<code>ul li.ativo a</code>	

▼ Ver gabarito

`h2` = 0-0-0-1 · `.card h2` = 0-0-1-1 · `#lojas .card h2` = 0-1-1-1 · `a:hover` = 0-0-1-1 · `ul li.ativo a` = 0-0-1-3.

Exercício 6: quem vence?

```
.menu a { color: white; }
.destaque { color: gold; }
<nav class="menu"><a class="destaque" href="#">Loja</a></nav>
```

▼ Ver gabarito

Branco. `.menu a` = 0-0-1-1 contra `.destaque` = 0-0-1-0. Correção: `.menu .destaque` (0-0-2-0).

Exercício 7: id contra classes

```
#topo { color: red; }
.a.b.c.d.e.f { color: blue; }
<p id="topo" class="a b c d e f">Oi</p>
```

▼ Ver gabarito

Vermelho. 0-1-0-0 vence 0-0-6-0: a coluna B pesa mais que qualquer número na coluna C.

Exercício 8: o atalho do inline

```
.botao { background: blue !important; }  
<button class="botao" style="background: red">Clique</button>
```

▼ Ver gabarito

Azul. O `!important` do autor vence até o estilo inline comum.

Parte 4 — Desafio de equipe

O CSS abaixo tem 3 problemas de predominância. Encontre e corrija sem usar `!important`.

```
#cabecalho .menu ul li a { color: white; }  
.botao { background: gray; }  
.botao--primario { background: blue; } /* vem antes de .botao no arquivo real */  
body main section div.container p { font-size: 14px; }  
.texto-grande { font-size: 24px; }
```

▼ Ver gabarito

1. Seletor longo com id (0-1-1-3): troque por `.menu-link { color: white; }`.
2. Ordem: `.botao--primario` deve ser escrito **depois** de `.botao`.
3. Seletor longo (0-0-1-5) impede `.texto-grande` de vencer: troque por `.paragrafo { font-size: 14px; }` e coloque `.texto-grande` depois.

O mapa: esqueleto de uma página profissional

Este é o documento central da trilha. Ele responde: **o que vem primeiro, o que fica dentro de quê, e por quê**. Leia o mapa de fora para dentro: **documento** → **body** → **blocos da página** → **seções** → **componentes**.

Mapa geral (de fora para dentro)

① `<!DOCTYPE html> + <html lang="pt-BR">`

O documento. Sempre a primeira linha. O lang ajuda leitor de tela e tradutor.

② `<head>`

Invisível. Informações sobre a página: charset, viewport, title, link do CSS.

③ `<body>`

Tudo o que o usuário vê. Aqui entram a fonte base (herança!) e o fundo.

④ `<header>`

Topo do SITE: logo + `<nav>`. Aparece em todas as páginas.

```
<a class="logo">
```

```
<nav class="menu">
```

```
<ul> <li> <a>
```

⑤ `<main>`

O conteúdo ÚNICO desta página. Só UM por página.

⑥ `<section class="hero">`

Faixa temática 1. Tem um título (h1 aqui).

```
<div class="container">
```

limita a largura (doc 03)

```
h1 · p · a.botao
```

⑦ componentes e texto

⑥ `<section class="secao-produtos">`

Faixa temática 2. Título h2.

```
<div class="container">
```



O mesmo mapa em código

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Bikcraft – Bikes sob medida</title>
  <link rel="stylesheet" href="estilo.css">
</head>
<body>

  <header class="cabecalho">
    <a class="logo" href="index.html">bikcraft</a>
    <nav class="menu" aria-label="Principal">
      <ul>
        <li><a href="bikes.html">Bikes</a></li>
        <li><a href="contato.html">Contato</a></li>
      </ul>
    </nav>
  </header>

  <div class="grid">
    <article class="card">
    <article class="card">
    <article class="card">
  </div>

  <aside>
    (opcional) conteúdo de apoio ao lado
  </aside>

  <footer class="rodape">
    ® Base do SITE: contatos, redes, direitos.
  </footer>

  <script src="...">
    Por último, para não travar a exibição.
  </script>
</body>
</html>
```

```
</nav>
</header>

<main>
  <section class="hero">
    <div class="container">
      <h1 class="hero-titulo">Sua bike, do seu jeito.</h1>
      <a class="botao botao--primario" href="#produtos">Ver bikes</a>
    </div>
  </section>

  <section class="produtos" id="produtos">
    <div class="container">
      <h2>Nossas bikes</h2>
      <div class="grid">
        <article class="card">...</article>
        <article class="card">...</article>
      </div>
    </div>
  </section>
</main>

<footer class="rodape">
  <div class="container">© 2026 Bikcraft</div>
</footer>

</body>
</html>
```

As 5 camadas, resumidas

Camada	Elementos	Pergunta que responde
1. Documento	html , head , body	É uma página? Como o navegador a lê?
2. Moldura do site	header , main , footer	Quais as 3 grandes áreas?
3. Seções	section , article , aside	Quais os assuntos dentro do conteúdo?
4. Estrutura e layout	div.container , div.grid	Como organizo largura e colunas?
5. Conteúdo e componentes	h1...h6 , p , a , img , .botao , .card	O que o usuário lê e clica?

Regra do mapa: as camadas 1 a 3 usam **tags semânticas** (dizem o que é). A camada 4 é o reino da **div** (só organiza). A camada 5 mistura os dois. Quem entende essa divisão nunca mais fica na dúvida entre div e section.

Árvore de decisão: div, main, section ou article?

Responda às perguntas **de cima para baixo**. A primeira resposta "sim" é a tag certa. A `div` é o **último recurso**: use quando nenhuma tag tem significado.

O fluxograma

É o topo do site (logo + menu), ou a abertura de um bloco?

sim ↓ (não: próxima pergunta)

`<header>`

É um grupo de links de navegação?

sim ↓

`<nav>`

É o conteúdo principal e único desta página?

sim ↓

`<main>` (só um por página)

Tem sentido sozinho, poderia ser publicado em outro lugar?
(notícia, produto, comentário, card)

sim ↓

`<article>`

É um grupo de conteúdo sobre UM tema, que deveria ter um título
(h2)?

sim ↓

`<section>`

É um conteúdo lateral, de apoio, que pode ser removido sem perder o texto principal?

sim ↓

`<aside>`

É a base do site ou de um bloco (contato, direitos)?

sim ↓

`<footer>`

Existe uma tag que diga o que é? (`ul` , `p` , `button` , `form` , `figure` ...)

sim ↓

Use **essa tag**

não ↓

`<div>`: só agrupa para o CSS (container, grid, linha de campos)

Pares que confundem

section ou div?

section: o grupo tem um **tema e um título**. Se você consegue nomear no sumário da página, é section.

div: só existe para o CSS (largura, grade, fundo).

Teste: "Se eu tirar o CSS, essa caixa ainda faz sentido como parte do documento?" Sim → section. Não → div.

section ou article?

article: peça **independente** e reaproveitável (card de produto, post, comentário).

section: **pedaço** de algo maior.

Teste: "Posso copiar para outro site e continua fazendo sentido?" Sim → article.

header ou section.hero?

header do site: topo com logo e menu. O **hero** é a primeira `section` do `main`.

Um `header` também pode abrir um `article` (título + data do post).

aside ou div.lateral?

aside: o conteúdo é **complementar** (links relacionados, anúncios, biografia).

div: uma coluna de layout que faz parte do texto principal.

Erros frequentes

Erro	Correção
Toda a página dentro de <code>div</code> ("divite")	Troque pela tag semântica correspondente
Dois <code>main</code> na mesma página	Só um. Os outros viram <code>section</code>
<code>section</code> sem título dentro	Dê um <code>h2</code> ou use <code>div</code>
<code>div</code> clicável com <code>onclick</code>	Use <code>button</code> (teclado e leitor de tela funcionam de graça)
<code>section</code> só para aplicar fundo	Isso é papel de <code>div</code> ou classe
<code>main</code> dentro de <code>section</code>	<code>main</code> envolve as seções, nunca o contrário

As semânticas não mudam o visual. `section`, `article` e `div` são todos caixas em bloco. A diferença é **significado**: para busca, leitor de tela e outros desenvolvedores. O visual você faz com classe.

Os elementos de página, um por um

Ficha técnica de cada elemento estrutural: o que é, quantos pode haver, onde fica, classe típica e erro comum.

Quadro-resumo

Tag	Quantos?	Fica dentro de	Classe típica
header	1 do site (+ 1 por article/section, se quiser)	body	cabecalho
nav	1 principal; outros com aria-label	header , footer	menu
main	1 só	body	raramente precisa
section	Quantas precisar	main	hero , produtos
article	Quantos precisar	section , main	card , post
aside	Quantos precisar	main ou ao lado	lateral
footer	1 do site (+ por article/section)	body	rodape
div	Sem limite	qualquer lugar	container , grid , campo

header

Cabeçalho do site: logo, menu, às vezes busca. **Não é o <head>** (esse é invisível e fica fora do body).

```
<header class="cabecalho">
  <a class="logo" href="/">bikcraft</a>
  <nav class="menu">...</nav>
</header>
```

CSS típico: `display: flex; justify-content: space-between; align-items: center;`

nav

Somente para **grupos principais de links** (menu, paginação, índice). Nem todo link solto precisa estar num `nav`. Estrutura profissional: lista dentro:

```
<nav class="menu" aria-label="Principal">
  <ul><li><a href="/bikes">Bikes</a></li> ...</ul>
</nav>
```

main

O coração da página: o conteúdo que **só existe aqui**. Não inclui cabeçalho, menu e rodapé repetidos em todas as páginas. Leitores de tela têm atalho que pula direto para ele.

section

Faixa temática com título. **Padrão da indústria:** `section` (faixa) + `div.container` (conteúdo).

```
<section class="lojas" aria-labelledby="titulo-lojas">
  <div class="container">
    <h2 id="titulo-lojas">Nossas lojas</h2>
    ...
  </div>
</section>
```

article

Item autônomo. Num catálogo, cada produto é um `article`; a lista deles vive numa `section`.

aside

Conteúdo relacionado, mas dispensável: barra lateral, caixa "leia também", propaganda, biografia do autor.

footer

Fecha o site (ou um bloco). Típico: endereço, redes, copyright, links legais.

div: o coringa

Caixa sem significado. Perfeita para três usos:

1. **Container:** limitar largura.
2. **Layout:** grid ou flex que agrupa filhos.
3. **Agrupamento visual:** linha de formulário, par rótulo+campo.

Outros elementos que completam o mapa

Tag	Para quê
<code>h1...h6</code>	Títulos, com hierarquia
<code>p</code>	Parágrafo
<code>ul</code> / <code>ol</code> / <code>li</code>	Listas (inclusive menus)
<code>figure</code> + <code>figcaption</code>	Imagem com legenda
<code>form</code> , <code>label</code> , <code>input</code> , <code>button</code>	Formulários
<code>a</code>	Link (leva a outro lugar)
<code>button</code>	Ação (faz algo na página)
<code>span</code>	Trecho de texto sem significado (<code>div</code> inline)
<code>img</code>	Imagem, sempre com <code>alt</code>

Link ou botão? Se muda de página → `a` . Se executa uma ação na página (enviar, abrir menu) → `button` . O visual pode ser idêntico: a classe cuida disso.

Qual classe começa? A ordem de escrita do CSS

A pergunta clássica: "por onde eu começo o CSS?". A resposta profissional é uma **ordem em camadas**, do mais geral ao mais específico. É a mesma lógica da cascata: **o que vem depois vence os empates**, então o mais genérico vem antes.

A escada das 8 camadas

- 1 Reset / normalização.** Zera diferenças entre navegadores: `* { box-sizing: border-box; }`, margens do `body`.
- 2 Variáveis (tokens).** Cores, fontes, espaços em um só lugar: `:root { --azul: #1b4f72; }`
- 3 Base (tags).** `body`, `h1...h6`, `p`, `a`, `img`. Aqui entra a herança: fonte, cor, line-height.
- 4 Estrutura.** `.container`, `.secao`. O esqueleto de largura e respiro.
- 5 Layout.** `.grid`, `.flex`, colunas. Como os blocos se organizam.
- 6 Componentes.** `.botao`, `.card`, `.menu`, `.rodape`, na ordem em que aparecem na página.
- 7 Modificadores e estados.** `.botao--primario`, `.is-open`. Depois do componente que alteram.
- 8 Utilitárias e media queries.** `.text-center`, `.hidden` e, por último, as media queries que ajustam tudo para telas maiores.

Mapa da ordem num único arquivo

```
/* 1. RESET */
* { box-sizing: border-box; }
body { margin: 0; }
```

```
/* 2. TOKENS */
:root { --azul: #1b4f72; --espaco: 16px; }

/* 3. BASE (tags) */
body { font-family: Arial, sans-serif; line-height: 1.6; color: #263238; }
a { color: inherit; }
img { max-width: 100%; display: block; }

/* 4. ESTRUTURA */
.container { max-width: 1100px; margin: 0 auto; padding: 0 var(--espaco); }
.secao { padding: 64px 0; }

/* 5. LAYOUT */
.grid { display: grid; gap: var(--espaco); grid-template-columns: repeat(auto-fit,
minmax(240px, 1fr)); }

/* 6. COMPONENTES */
.cabecalho { ... }
.botao { ... }
.card { ... }
.rodape { ... }

/* 7. MODIFICADORES E ESTADOS */
.botao--primario { ... }
.menu.is-open { ... }

/* 8. UTILITÁRIAS E RESPONSIVO */
.text-center { text-align: center; }
.hidden { display: none; }
@media (min-width: 768px) { ... }
```

Por que nesta ordem?

Camada	Motivo de vir nessa posição
Reset e base primeiro	São o piso. Qualquer classe precisa vencê-los sem esforço.
Estrutura antes dos componentes	Quase todo componente vive dentro de um container.
Modificador depois do componente	Mesmo peso 0-0-1-0, vence quem vem por último.
Utilitárias no fim	São a "palavra final" do desenvolvedor.
Media queries no fim	Mesmo peso que a regra base; precisam vir depois para sobrescrevê-la.

Mobile-first: a base é o celular

As regras fora de media query descrevem o **celular**. Telas maiores entram por `min-width`:

```
.grid { grid-template-columns: 1fr; } /* celular */
@media (min-width: 768px) {
  .grid { grid-template-columns: repeat(3, 1fr); } /* tablet para cima */
}
```

E no HTML, qual classe aparece primeiro?

Escreva as classes em ordem de **geral** → **específico**. A ordem no atributo `class` **não** muda a cascata (quem manda é a ordem no CSS), mas é uma convenção que facilita a leitura:

```
<button class="botao botao--primario botao--grande mt-3">
  componente  variação      variação  utilitária
```

Pegadinha: a ordem das classes dentro do `class="..."` do HTML **não importa** para quem vence. Quem decide é a **posição da regra no arquivo CSS**, depois da especificidade.

Nomenclatura (BEM) e organização de arquivos

Nomes ruins causam conflitos; nomes bons se explicam sozinhos. **BEM** é a convenção mais usada do mercado para nomear classes. Junto com uma estrutura de pastas simples, é o que faz um projeto crescer sem virar bagunça.

BEM: Bloco, Elemento, Modificador

Parte	Sintaxe	Exemplo	Significa
Bloco	<code>bloco</code>	<code>card</code>	A peça independente
Elemento	<code>bloco__elemento</code>	<code>card__titulo</code>	Uma parte que só existe dentro do bloco
Modificador	<code>bloco--variacao</code>	<code>card--destaque</code>	Uma variação ou estado

```
<article class="card card--destaque">
  
  <h3 class="card__titulo">Urbana</h3>
  <a class="card__link botao" href="#">Ver</a>
</article>
```

```
.card { ... }
.card__imagem { ... }
.card__titulo { ... }
.card--destaque { border-color: gold; }
```

Por que BEM resolve problema de predominância

- Todo seletor é **uma classe só** (0-0-1-0): nunca há briga de peso.
- Sem seletores aninhados como `.card h3`, mexer em um componente não quebra outro.
- O nome **informa a hierarquia**: `card__titulo` pertence ao `card`.

Não aninhe os nomes: escreva `card__titulo`, não `card_corpo__titulo`. Elemento é sempre filho do bloco, mesmo que esteja no meio de outras divs.

Seu projeto, sem BEM rígido

Na sala usamos a versão leve: `hero` , `hero-titulo` , `card-loja` , `coluna-dados` . A ideia é a mesma: **prefixo = dono**. Importante é ser **consistente**.

Regras gerais de nomeação

- 1 **Minúsculas e hífens** (*kebab-case*), nunca espaço ou acento.
- 2 **Nome por função, não por aparência**: `botao-primario` , não `botao-azul` .
- 3 **Português ou inglês, mas um só** em todo o projeto.
- 4 **Nada de numeração** (`caixa2` , `div3`): o nome deve dizer o conteúdo.
- 5 **Prefixo para estado**: `is-` / `has-` (controlados por JS).
- 6 **Prefixo para JS**: `js-` quando a classe existe só para script, nunca para estilo.

Organização de arquivos

Projeto pequeno (o nosso caso)

```
projeto/  
├─ index.html  
├─ contato.html  
├─ estilo.css           ← tudo, na ordem do documento 20  
├─ script.js  
└─ assets/  
    ├─ bike-1.svg  
    └─ logo.svg
```

Projeto médio

```
projeto/  
├─ index.html  
├─ css/  
│   └─ 1-reset.css
```

```
|   |─ 2-variaveis.css
|   |─ 3-base.css
|   |─ 4-layout.css
|   |─ 5-componentes.css
|   |─ 6-utilitarias.css
|─ js/
|─ assets/
|   |─ img/
|   |─ fonts/
```

Os números no nome dos arquivos refletem a **ordem do documento 20**. No HTML, os `<link>` seguem essa ordem.

Padrão de comentários

```
/* =====
COMPONENTE: CARD
Usado em: catálogo, lojas
===== */
.card { ... }
```

Teste de legibilidade: um colega consegue achar, em 10 segundos, onde mudar a cor do botão principal? Se sim, sua organização está boa.

Página completa anotada (Bikcraft)

Juntando tudo: uma página inteira comentada **linha a linha**, com a justificativa de cada escolha. É a estrutura da página de contato que a turma construiu em `02-10/4-pagina-contato-final-gabarito.html`. **Este é o padrão correto:** `header`, `main` com as seções dentro, `footer`. Os arquivos da atividade 4 seguem exatamente esse esqueleto.

1. HTML anotado

```
<!DOCTYPE html>                                <!-- avisa: HTML5 moderno -->
<html lang="pt-BR">                             <!-- idioma: leitor de tela e tradutor -->
<head>
  <meta charset="UTF-8">                        <!-- acentos corretos -->
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
                                              <!-- sem isto o celular finge ser desktop -->
  <title>Contato – Bikcraft</title>             <!-- aba do navegador e Google -->
  <link rel="stylesheet" href="estilo.css">
</head>
<body>

  <!-- CAMADA 2: moldura -->
  <header class="cabecalho">                    <!-- topo do SITE -->
    <a class="logo" href="index.html">bikcraft</a>
    <nav class="menu" aria-label="Principal">
      <a href="index.html">Home</a>
      <a href="bikes.html">Bikes</a>
      <a href="contato.html" aria-current="page">Contato</a>
    </nav>
  </header>

  <main>                                         <!-- conteúdo ÚNICO desta página -->

    <!-- CAMADA 3: seção hero -->
    <section class="hero">                      <!-- faixa de destaque -->
      <p class="hero-chamada">Respostas em até 24h</p>
      <h1 class="hero-titulo">Nosso contato</h1> <!-- UM h1 por página -->
    </section>

    <!-- seção de duas colunas -->
    <section class="secao-contato">
      <div class="coluna-dados"> ... </div>      <!-- div: só organiza layout -->
      <form class="coluna-formulario">          <!-- form: tag certa, não div -->
        <div class="campo">
          <label for="nome">Nome</label>
```

```

        <input id="nome" type="text" required> <!-- label ligado por for/id -->
    </div>
    <button class="botao" type="submit">Enviar</button>
</form>
</section>

<!-- lista de cards: article dentro de section -->
<section class="lojas">
    <article class="card-loja"> ... </article>
    <article class="card-loja"> ... </article>
</section>
</main>

<footer class="rodape"> ... </footer> <!-- base do SITE -->

<a class="topo" href="#" aria-label="Voltar ao topo">↑</a>
<script src="script.js"></script> <!-- último: não trava a página -->
</body>
</html>

```

2. CSS anotado (na ordem do documento 20)

```

/* RESET + TOKENS */
* { box-sizing: border-box; }
:root { --azul: #1b4f72; --gap: 16px; }

/* BASE: herança trabalhando — fonte e cor declaradas UMA vez */
body { margin: 0; font-family: Arial, sans-serif; line-height: 1.6; color: #263238; }
a { color: inherit; }
input, button { font: inherit; }

/* ESTRUTURA / LAYOUT */
.secao-contato { display: flex; flex-direction: column; gap: var(--gap); padding: 40px 16px; }
@media (min-width: 768px) {
    .secao-contato { flex-direction: row; } /* mobile-first: duas colunas só no tablet */
    .coluna-dados, .coluna-formulario { flex: 1; }
}

/* COMPONENTES (classe simples, peso 0-0-1-0) */
.cabecalho { display: flex; justify-content: space-between; align-items: center; padding: 12px 16px; }
.campo { display: flex; flex-direction: column; margin-bottom: var(--gap); }
.botao { padding: 10px 20px; background: var(--azul); color: #fff; border: 0; border-radius: 6px; }
.card-loja { border: 1px solid #cfd8dc; border-radius: 8px; padding: 16px; }

```

```
/* ESTADOS */
.botao:hover { background: #123049; }
.menu a[aria-current="page"] { font-weight: bold; }
```

3. Mapa das decisões

Escolha	Regra aplicada	Doc
<code>header</code> , <code>main</code> , <code>footer</code> como moldura	Camada 2 do mapa	17
<code>section</code> para hero, contato, lojas	Cada faixa tem um tema	18
<code>article</code> para cada loja	Peça independente	18
<code>div.campo</code>	Agrupar só para o CSS	18
<code>form</code> , <code>label</code> , <code>button</code>	Existe tag com significado	19
Fonte só no <code>body</code>	Herança	10
<code>a { color: inherit }</code>	Link não herda a cor sozinho	11
Tudo com classe, sem id no CSS	Baixa especificidade	14, 15
Media query depois da regra base	Mesmo peso, vence a última	12, 20

Confira você mesmo: abra `02-10/4-pagina-contato-final-gabarito.html` ao lado deste documento e localize, na página da turma, cada camada do mapa: `header` , `main` , as três `section` , os `article` , o `footer` . Todas estão lá, na mesma ordem.

Do básico ao avançado e checklist profissional

Para fechar: a **escada de evolução** (onde você está, qual o próximo degrau) e um **checklist** para revisar qualquer página antes de entregar.

A escada em 5 níveis

Nível 1 — Raiz: a página funciona

Sabe: `html/head/body` , títulos, parágrafos, links, imagens, listas. Cria classes e aplica cor e fonte.

Típico: usa só `div` e `span` . Estiliza por tag.

Próximo degrau: trocar `div` por tags semânticas (docs 17–19).

Nível 2 — Estrutura: a página tem esqueleto

Sabe: `header / main / section / footer` , `container`, uma classe por função, hierarquia `h1→h2→h3`.

Próximo degrau: dominar layout (`flex`, `grid`) e herança (docs 04, 10, 11).

Nível 3 — Layout: a página se adapta

Sabe: `flexbox` e `grid`, `rem / % / vw` , `mobile-first` com `min-width` , imagens fluidas.

Próximo degrau: entender a cascata a fundo e sair do "tentativa e erro" (docs 12–15).

Nível 4 — Cascata: o CSS é previsível

Sabe: calcula especificidade de cabeça, não usa `id` nem `!important` para estilizar, ordena o CSS em camadas, depura com DevTools.

Próximo degrau: nomenclatura e arquitetura (doc 21).

Nível 5 — Arquitetura: o projeto escala

Sabe: BEM, variáveis CSS (tokens), componentes reutilizáveis, arquivos organizados, acessibilidade (rótulos, foco, contraste, `alt`), escolhe biblioteca (Bootstrap/Tailwind) por critério.

Depois disso: `@layer`, `:where()`, `clamp()`, container queries, CSS com JavaScript e frameworks (React).

Checklist de revisão profissional

HTML

- ☐ Começa com `<!DOCTYPE html>` e `<html lang="pt-BR">`
- ☐ `charset`, `viewport` e `title` presentes no `head`
- ☐ Exatamente **um** `<main>` e **um** `<h1>`
- ☐ Títulos em ordem (`h1` → `h2` → `h3`), sem pular nível
- ☐ Moldura com `header` / `main` / `footer`
- ☐ `section` só com tema e título; `article` só para peça independente
- ☐ `div` usada apenas quando nenhuma tag serve
- ☐ Toda `img` tem `alt`; todo `input` tem `label`
- ☐ Link para navegar, `button` para agir
- ☐ Indentação consistente, tags fechadas

CSS

- ☐ Reset e base primeiro; componentes no meio; utilitárias e media queries no fim
- ☐ Fonte e cor definidas **uma vez** no `body` (herança)
- ☐ Seletores curtos: quase tudo é uma classe só
- ☐ Sem `id` no CSS, sem `style=""` inline, `!important` só justificado
- ☐ Nomes por função, minúsculas com hífen, língua única
- ☐ Valores repetidos viraram variável (`--azul`)
- ☐ Mobile-first (`min-width`)
- ☐ Nenhum `outline: none` sem substituto de foco

Teste final (2 minutos)

- 1 Reduza a janela até 320px. Nada sai da tela? Sem barra de rolagem horizontal?

2 Navegue só com a tecla **Tab**. Dá para ver onde está o foco?

3 Aumente o zoom do navegador para 200%. O layout aguenta?

4 Abra o DevTools no elemento mais complexo. Quantas propriedades riscadas?
Quanto menos, melhor.

Cola de bolso: perguntas que resolvem 90% das dúvidas

"Qual tag uso?"

Existe uma tag com o significado? Use.
Senão, `div`. (doc 18)

"Meu estilo não aparece"

Seletor acha o elemento? Alguém mais específico? Empate com ordem errada?
(docs 13–15)

"Onde declaro a fonte?"

No `body`. Os filhos herdam. (docs 10–11)

"Qual classe escrevo primeiro?"

Do geral para o específico: base →
estrutura → componente → modificador
→ utilitária. (doc 20)

"Como nomeio?"

O que é, não como parece. Prefixo =
dono. (docs 02, 21)

"Flex ou grid?"

Uma dimensão: flex. Duas: grid. (doc 04)

Parabéns! Quem domina esses 23 documentos escreve HTML e CSS que outro profissional consegue ler, manter e expandir. Esse é o critério que o mercado usa.